



Algorithms and Data Structures

Sujeong Kim



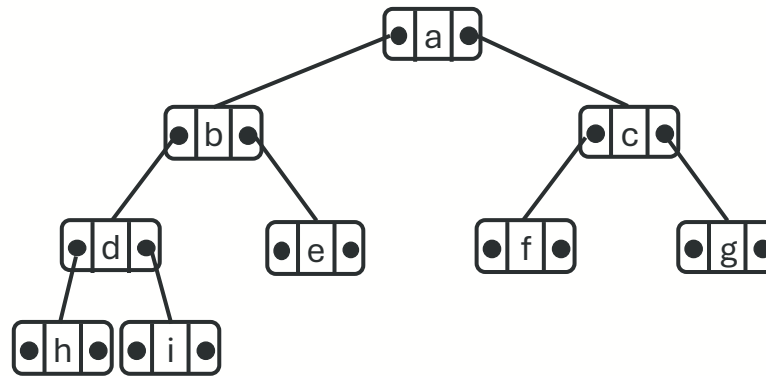
UNIVERSITY OF
MARYLAND

Heap and Heap Sort, Priority Queue

Slide Credit: These slides are adapted from materials by Prof. Manocha and Prof. Ayhan, with modifications and additions by Sujeong Kim.

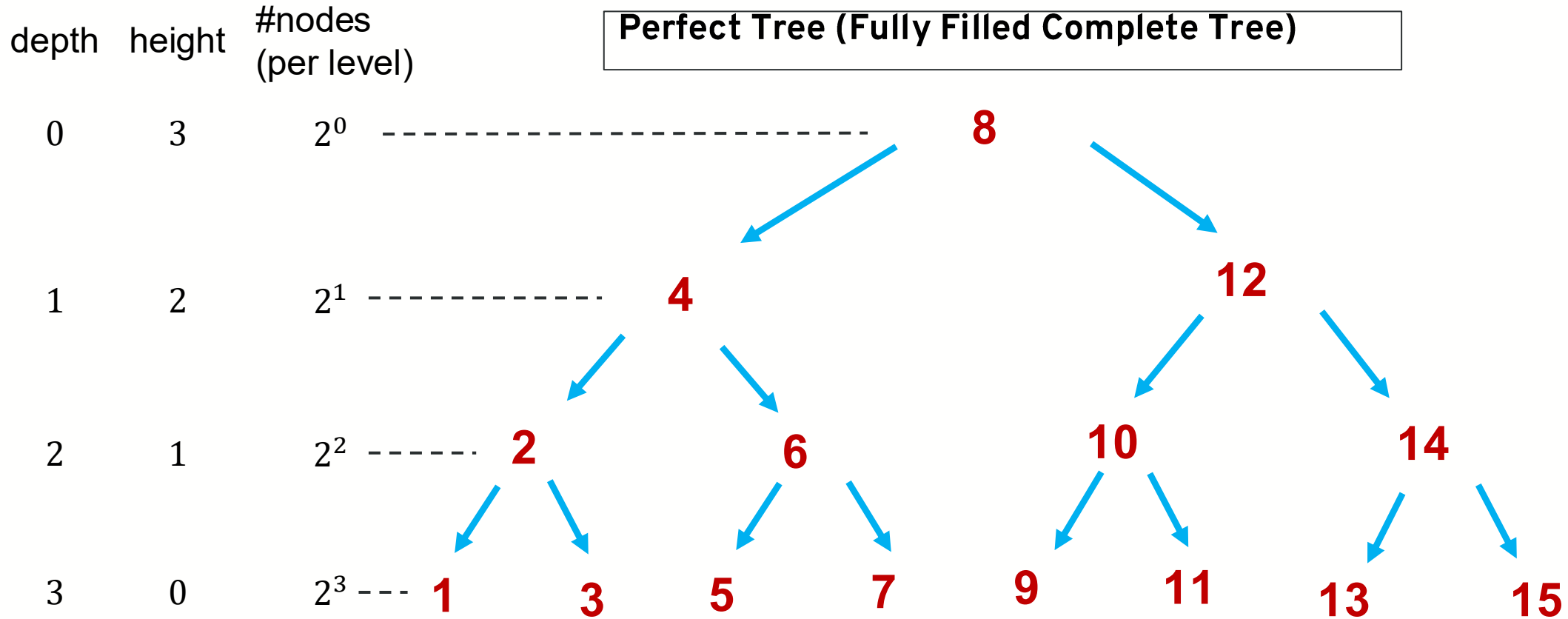
(Recall) Complete Binary Tree

- A complete binary tree is completely filled, with the possible exception of the bottom level, which is filled from left to right
- The depth of a node is the number of edges from the root
- The depth of a tree is the depth of the deepest node



complete binary tree

Recall BT Depth, Height vs. #Nodes



Total number of nodes = $2^0 + 2^1 + 2^2 + \dots + 2^h = 1 \cdot \frac{1-2^{h+1}}{1-2} = 2^{h+1} - 1$

$sum = a \frac{1 - r^n}{1 - r}$
 ($a = 2^0, r = 2, n = h + 1$)

Recall Complete BT Depth, Height vs. #Nodes

Height of
the tree

#nodes
(size of the tree)

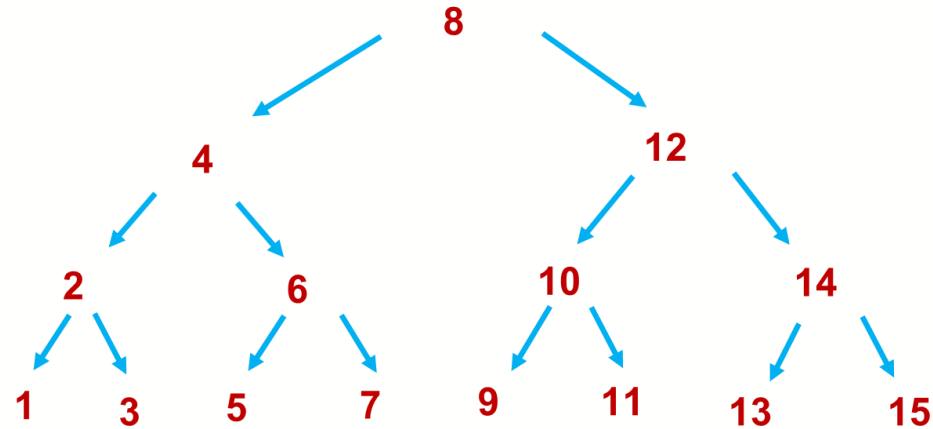
0 1

1 2, 3

2 4, 5, 6, 7

3 8 to 15

h 2^h to $2^{h+1} - 1$



*Complete binary tree
height h*

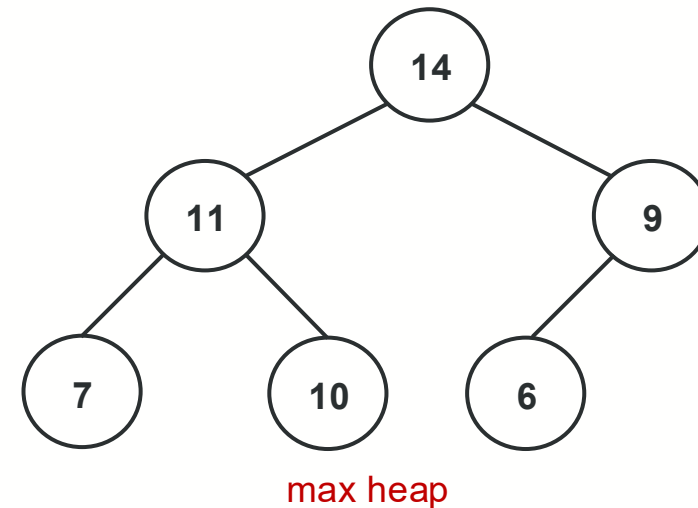
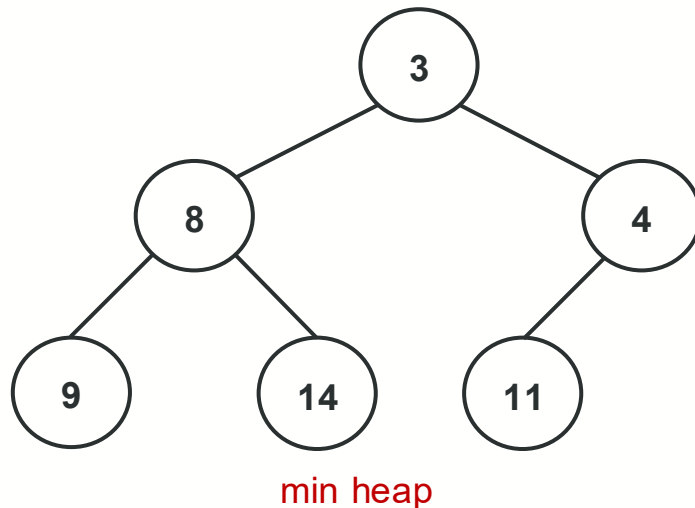
number of nodes n
 $2^h \leq n \leq 2^{h+1} - 1$

*Complete binary tree
of size n*

height of tree
 $\lfloor \log_2 n \rfloor$

Heap

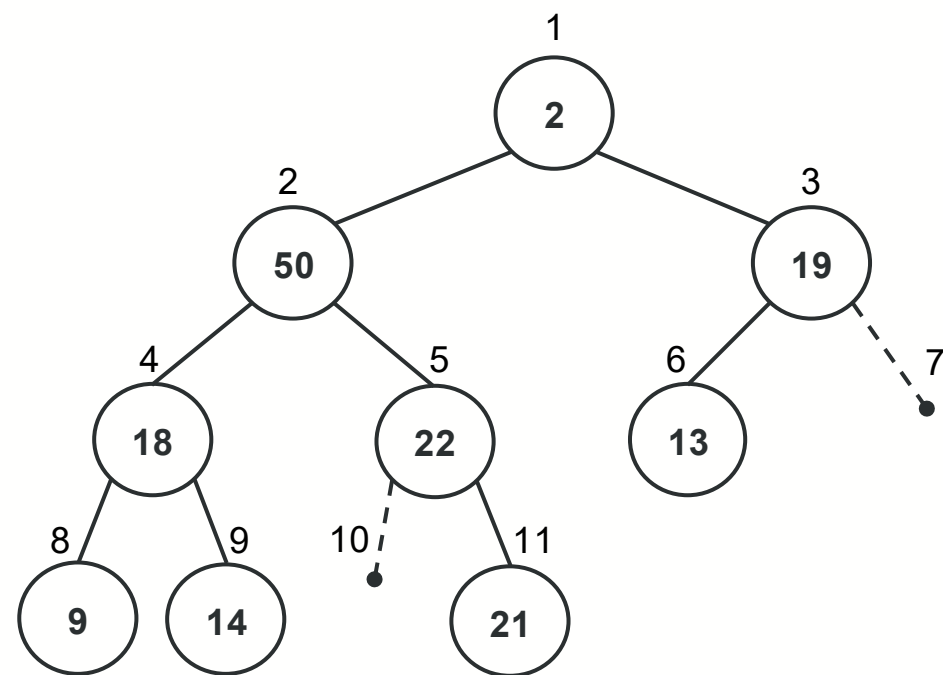
- A (binary) heap is a complete binary tree in which each node satisfies the heap ordering property
 - min-heap property: the value of each node is greater than or equal to the value of its parent, with the minimum-value element at the root
 - max-heap property: the value of each node is less than or equal to the value of its parent, with the maximum-value element at the root



Heap (2/2)

- It is not a sorted structure and can be considered as partially ordered
- Since a heap is a complete binary tree, it has a smallest possible height
 - A heap with n nodes always has $O(\log n)$ height
- A heap is usually implemented as an array
- A max-heap can be converted to a min heap (and vice versa) by simply inverting the comparison operation used in the heap property
- In this lecture, we are going to use examples of Max-heaps

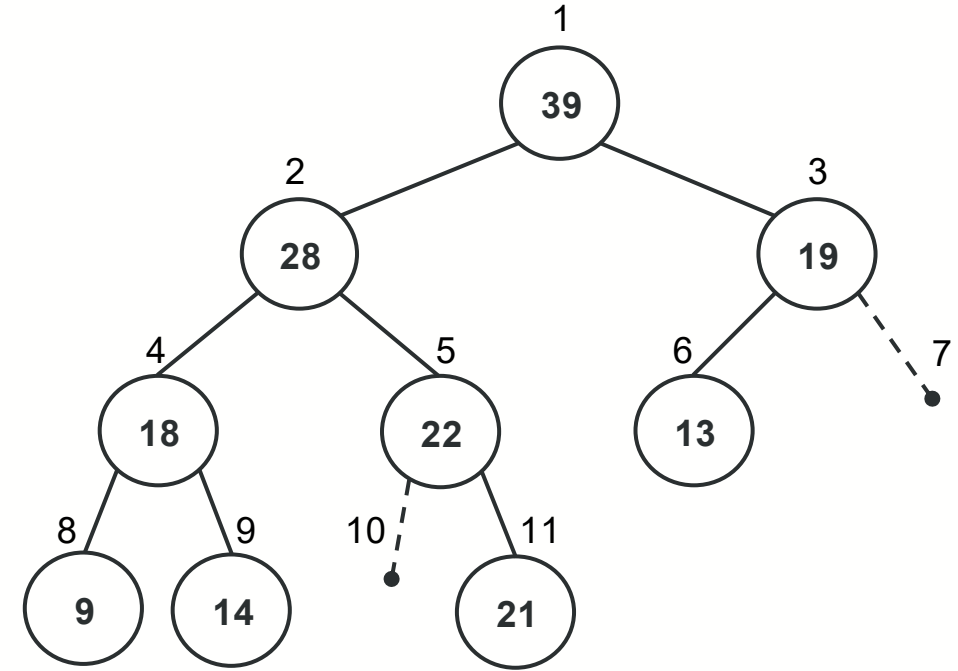
Array Representation of Binary Trees



1	2	3	4	5	6	7	8	9	10	11
2	50	19	18	22	13	-	9	14	-	21

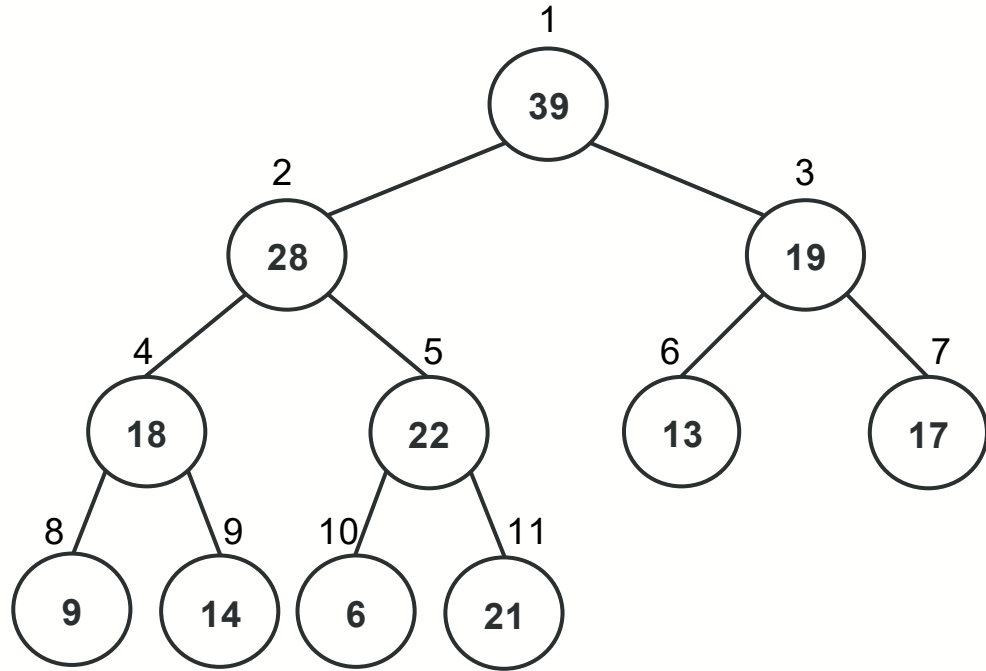
Basic Operations

- Parent(i)
return $\lfloor i/2 \rfloor$
- Left(i)
return $2i$
- Right(i)
return $2i+1$



1	2	3	4	5	6	7	8	9	10	11
39	28	19	18	22	13	-	9	14	-	21

Array Representation of a Heap



1	2	3	4	5	6	7	8	9	10	11
39	28	19	18	22	13	17	9	14	6	21

Max Heap: $A[\text{Parent}(i)] \geq A[i]$

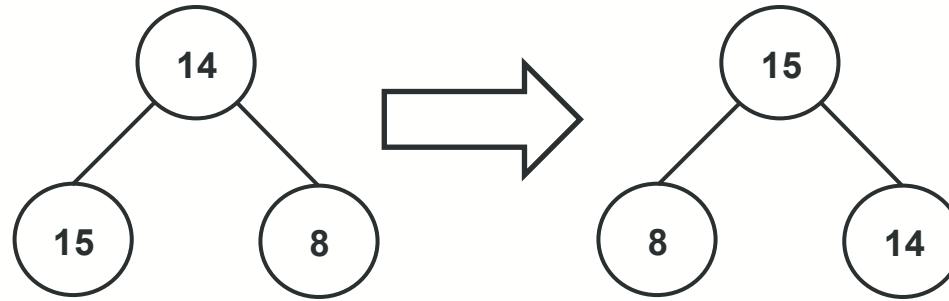
Algorithms maintain heap property as data is added/removed

Height

- Height of a node in a tree: the number of edges on the longest simple downward path from the node to a leaf
- Height of a tree: the height of the root
- Height of the tree for a heap: $\Theta(\lg n)$
 - n is the size of the tree (number of nodes)
 - Basic operations on a heap run in $O(\lg n)$ time

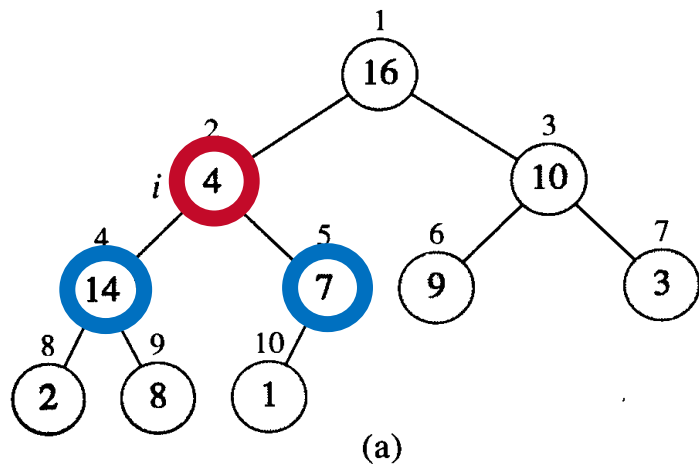
Maintaining Heap Property

- We can set a heap property to a node by exchanging a node(s) value with the value of its child

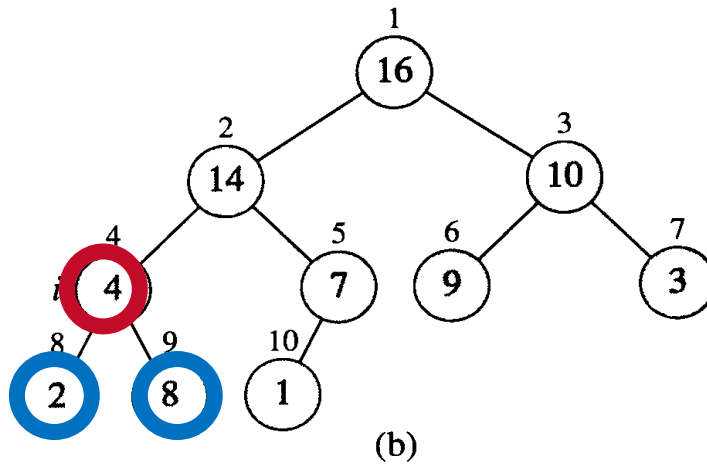


setting max heap
property

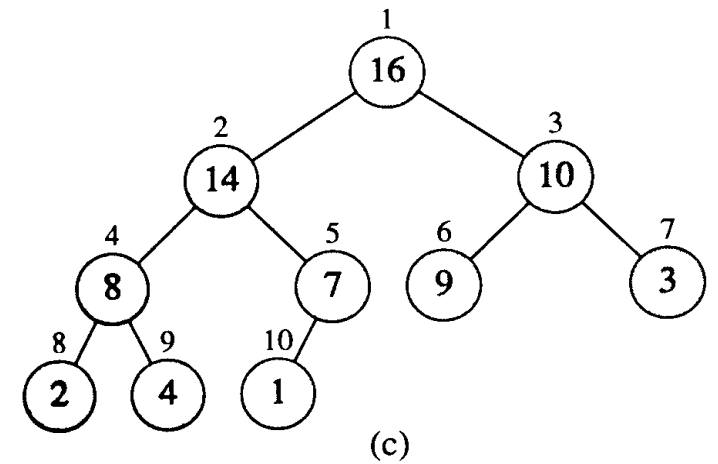
Maintaining Heap Property - Heapify



1	2	3	4	5	6	7	8	9	10
16	4	10	14	7	9	3	2	8	1



1	2	3	4	5	6	7	8	9	10
16	14	10	4	7	9	3	2	8	1



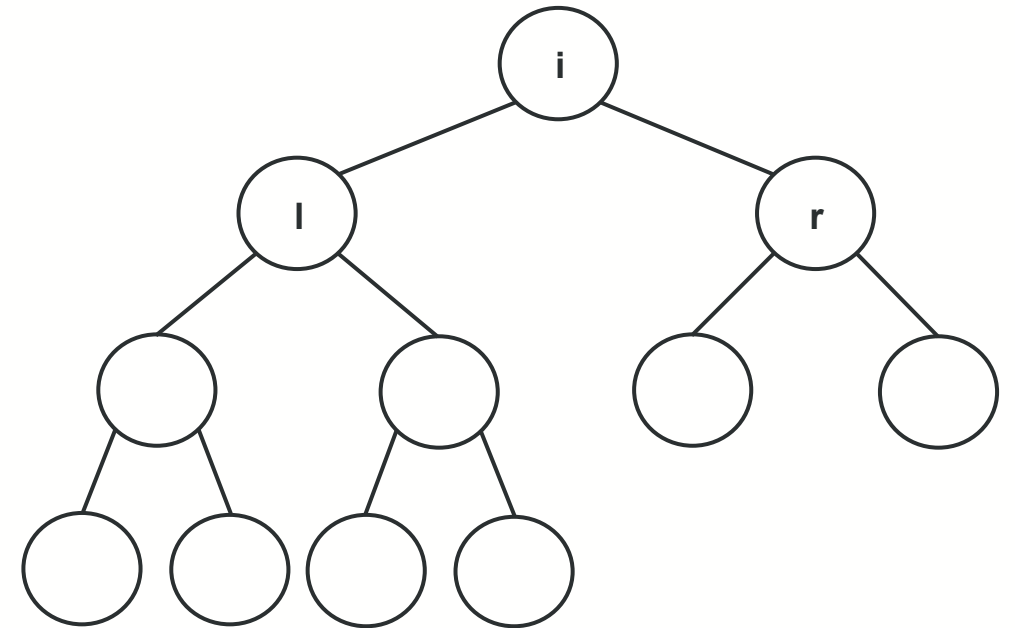
1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1

Heapify (A, i)

1. $l \leftarrow \text{left}(i)$

2. $r \leftarrow \text{right}(i)$

- 3.
- 4.
5. Find the largest value among node i and its child(ren)
- 6.
- 7.
8. if node i is not the largest (i.e., violates Max Heap property)
9. Swap it with the larger child
10. Repeat the process for the sub-tree affected by the swap



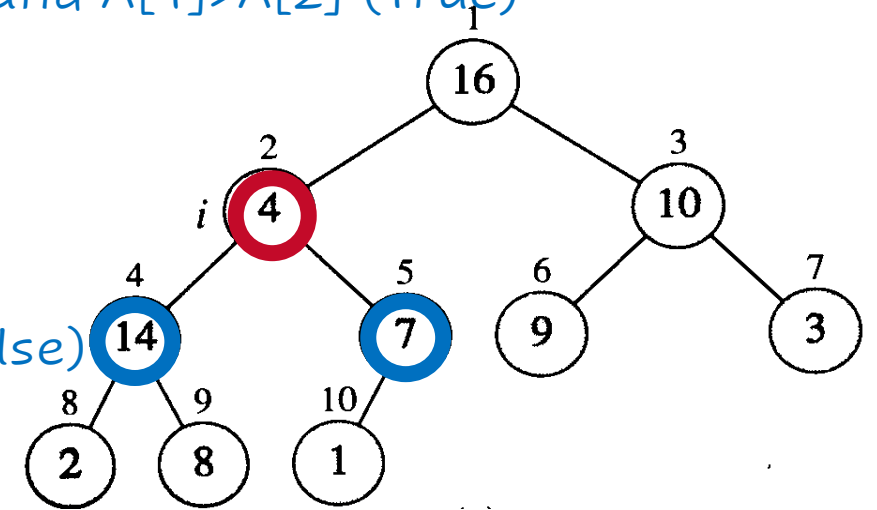
Heapify (A, i)

1. $l \leftarrow \text{left}(i)$
2. $r \leftarrow \text{right}(i)$
3. if $l \leq \text{heap-size}[A]$ and $A[l] > A[i]$
4. then $\text{largest} \leftarrow l$
5. else $\text{largest} \leftarrow i$
6. if $r \leq \text{heap-size}[A]$ and $A[r] > A[\text{largest}]$
7. then $\text{largest} \leftarrow r$
8. if $\text{largest} \neq i$
9. then exchange $A[i] \leftrightarrow A[\text{largest}]$
10. Heapify(A, largest)

Heapify (A, i)

Heapify(A, 2)
i=2

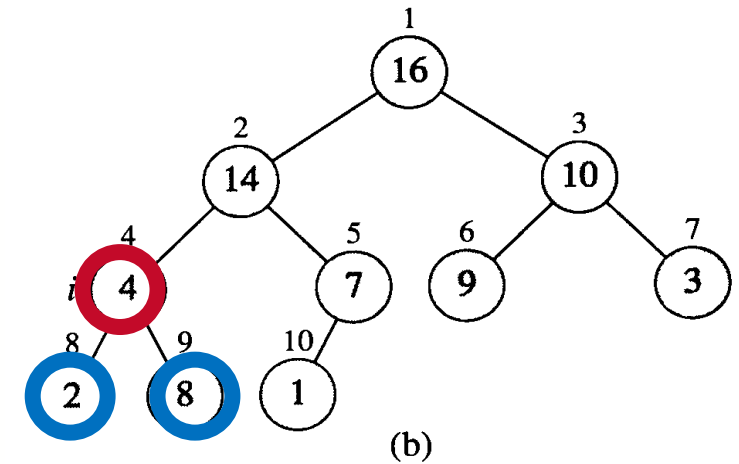
1. $l \leftarrow \text{left}(i)$ $l=4$
2. $r \leftarrow \text{right}(i)$ $r=5$
3. if $l \leq \text{heap-size}[A]$ and $A[l] > A[i]$ $l \leq 10$ (True) and $A[4] > A[2]$ (True)
4. then $\text{largest} \leftarrow l$ $\text{largest} \leftarrow l=4$
5. else $\text{largest} \leftarrow i$
6. if $r \leq \text{heap-size}[A]$ and $A[r] > A[\text{largest}]$ $r \leq 10$ (True) and $A[5] > A[4]$ (False)
7. then $\text{largest} \leftarrow r$
8. if $\text{largest} \neq i$ (True)
9. then exchange $A[i] \leftrightarrow A[\text{largest}]$ $\text{swap}(A[2], A[4])$
10. Heapify(A, largest) $\text{Heapify}(A, 4)$



1	2	3	4	5	6	7	8	9	10
16	4	10	14	7	9	3	2	8	1
	i		l	r					

Heapify (A, i) *Heapify(A, 4)*

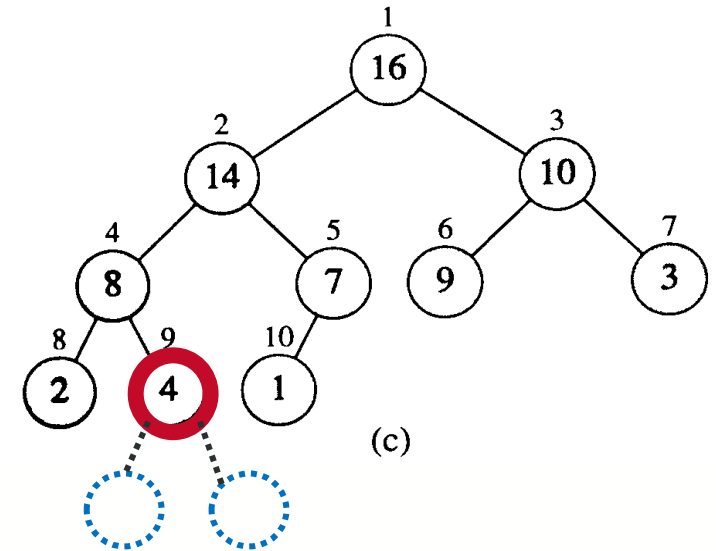
1. $l \leftarrow \text{left}(i)$ *$l=8$*
2. $r \leftarrow \text{right}(i)$ *$r=9$*
3. if $l \leq \text{heap-size}[A]$ and $A[l] > A[i]$ *$l \leq 10$ (True) and $A[l] > A[i]$ (False)*
4. then $\text{largest} \leftarrow l$
5. else $\text{largest} \leftarrow i$ *$\text{largest} \leftarrow i=4$*
6. if $r \leq \text{heap-size}[A]$ and $A[r] > A[\text{largest}]$
7. then $\text{largest} \leftarrow r$ *$r \leq 10$ (True) and $A[9] > A[4]$ (True)
 *$\text{largest} \leftarrow r=9$**
8. if $\text{largest} \neq i$ *(True)*
9. then exchange $A[i] \leftrightarrow A[\text{largest}]$
10. $\text{Heapify}(A, \text{largest})$ *$\text{Heapify}(A, 9)$*



1	2	3	4	5	6	7	8	9	10
16	14	10	4	7	9	3	2	8	1
			<i>i</i>				<i>l</i>		<i>r</i>

Heapify (A, i) Heapify(A, 9)

1. $l \leftarrow \text{left}(i)$ $l=18$
2. $r \leftarrow \text{right}(i)$ $r=19$
3. if $l \leq \text{heap-size}[A]$ and $A[l] > A[i]$ $l \leq 10$ (False)
4. then $\text{largest} \leftarrow l$
5. else $\text{largest} \leftarrow i$ $\text{largest} \leftarrow i=9$
6. if $r \leq \text{heap-size}[A]$ and $A[r] > A[\text{largest}]$
7. then $\text{largest} \leftarrow r$ $r \leq 10$ (False)
8. if $\text{largest} \neq i$ (False) $\rightarrow \text{largest} == i$
9. then exchange $A[i] \leftrightarrow A[\text{largest}]$
10. Heapify(A, largest)



1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1
									i

Heapify(A,i) - Assumptions

- Both children of node i already satisfy the heap property
 - Left subtree is a valid heap
 - Right subtree is a valid heap
- Only the node at index i may violate the heap property
- Heap is stored in an array, using the standard child index formulas
 - $\text{left}(i) = 2i$, $\text{right}(i) = 2i + 1$
- We are working with a complete binary tree (so children exist or don't exist in predictable positions)

Running Time for Heapify(A, i)

```
1.  $l \leftarrow \text{left}(i)$   
2.  $r \leftarrow \text{right}(i)$   
3. if  $l \leq \text{heap-size}[A]$  and  $A[l] > A[i]$   
4.   then  $\text{largest} \leftarrow l$   
5.   else  $\text{largest} \leftarrow i$   
6. if  $r \leq \text{heap-size}[A]$  and  $A[r] > A[\text{largest}]$   
7.   then  $\text{largest} \leftarrow r$   
8. if  $\text{largest} \neq i$   
9.   then exchange  $A[i] \leftrightarrow A[\text{largest}]$ 
```

$T(i) =$

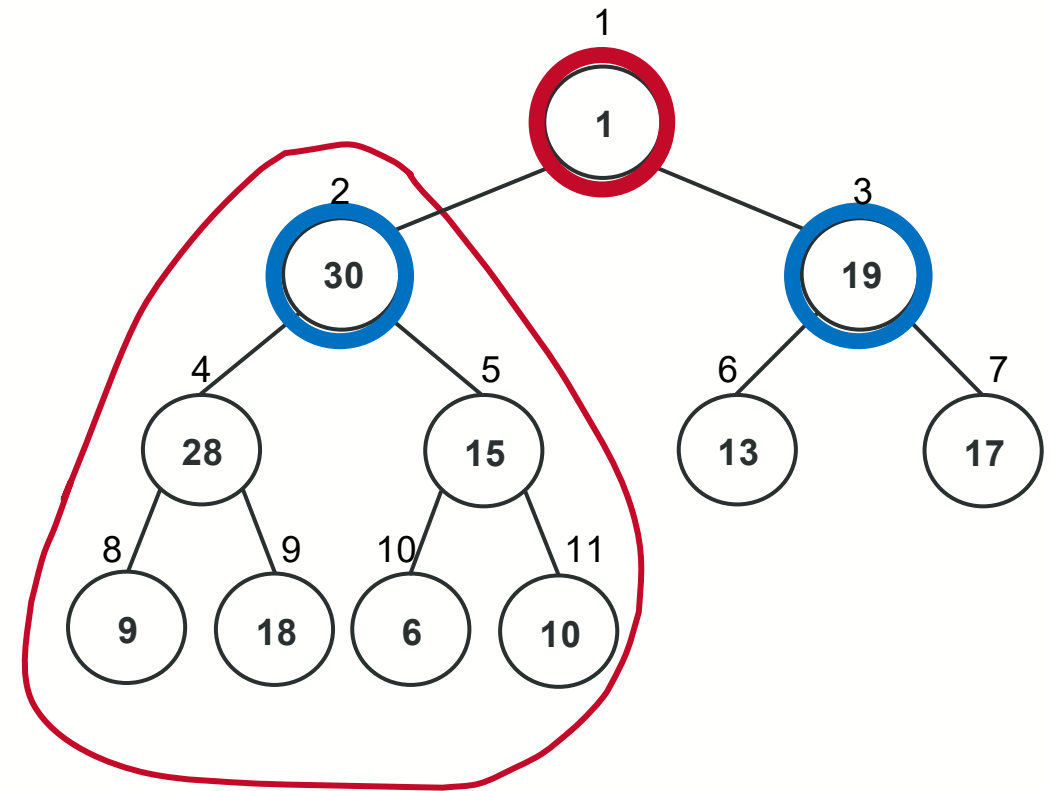
$\Theta(1) +$

```
10.   Heapify( $A, \text{largest}$ )
```

$T(\text{largest})$

Running Time for Heapify(A, n)

- So, $T(n) = T(\text{largest}) + \Theta(1)$
- Worst case: size of largest subtree
 - $\text{largest} \leq \frac{2}{3}n$
 - If the last row is half full, the larger child subtree contains at most $\frac{2}{3}n$ nodes
- $T(n) \leq T(\frac{2}{3}n) + \Theta(1)$
 $\Rightarrow T(n) = O(\lg n)$

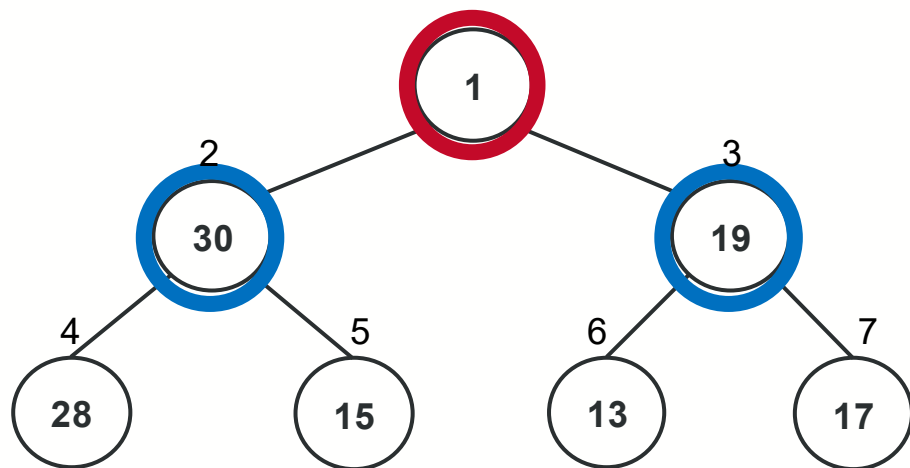


$n=11$, $\text{largest}=7$

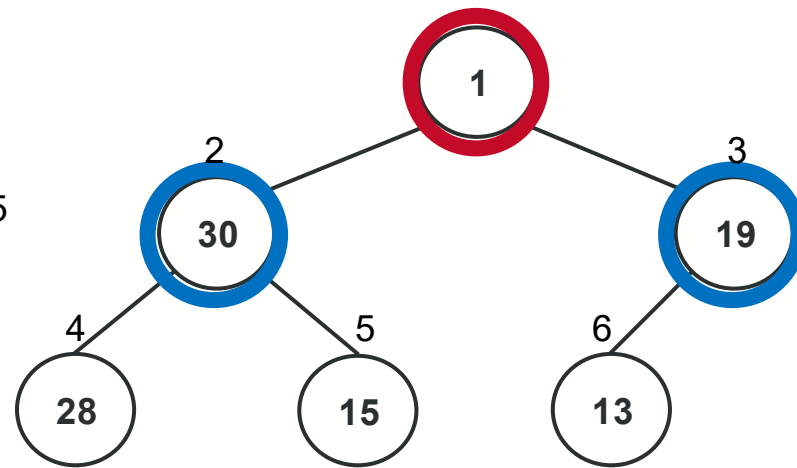
$$7 \cong \frac{2}{3} \cdot 11$$

$$\frac{2}{3} \approx 0.67$$

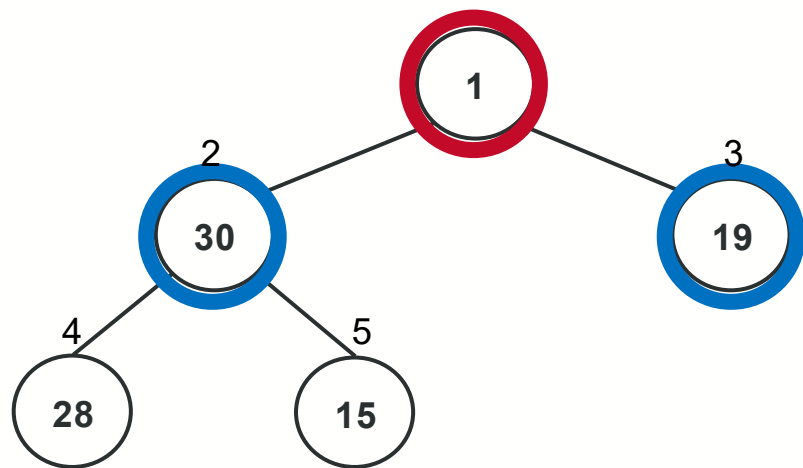
n=7
largest=3
largest/n=0.43



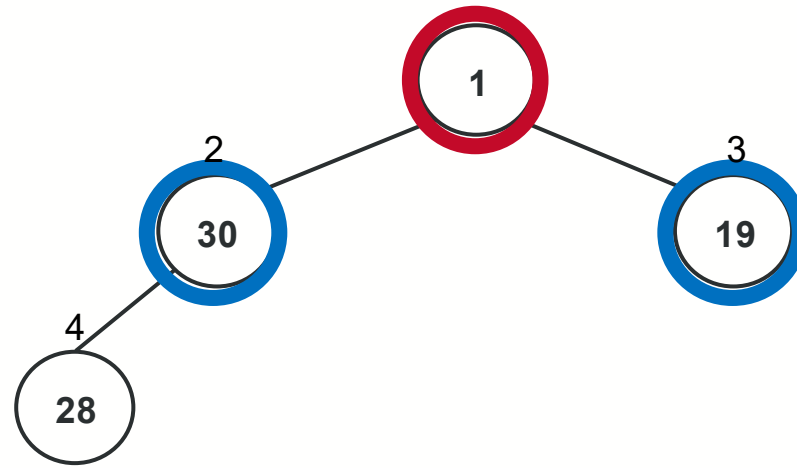
n=6
largest=3
largest/n=0.5



n=5
largest=3
largest/n=0.6



n=4
largest=2
largest/n=0.5

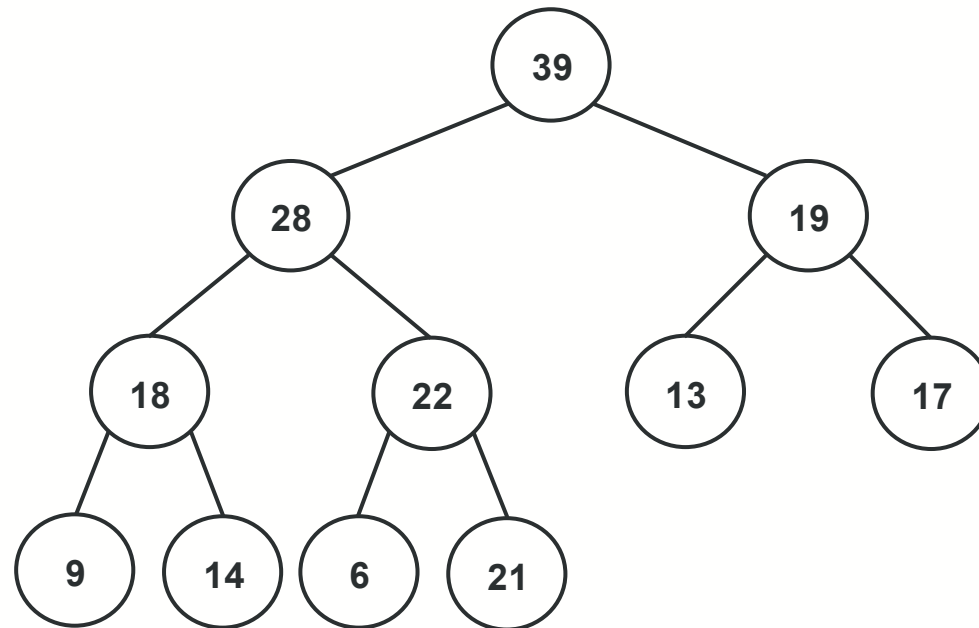


Running Time for Heapify(A, n)

- Alternately, Heapify takes $O(h)$ where h is the height of the node where Heapify is applied
- For a complete binary tree, this height is at most $O(\lg n)$, so Heapify also runs in $O(\lg n)$ time

A Heapified Binary Tree

- The fact that a heapified binary tree doesn't mean sorted is remarkable
- The process of heapification doesn't change the shape of the binary tree

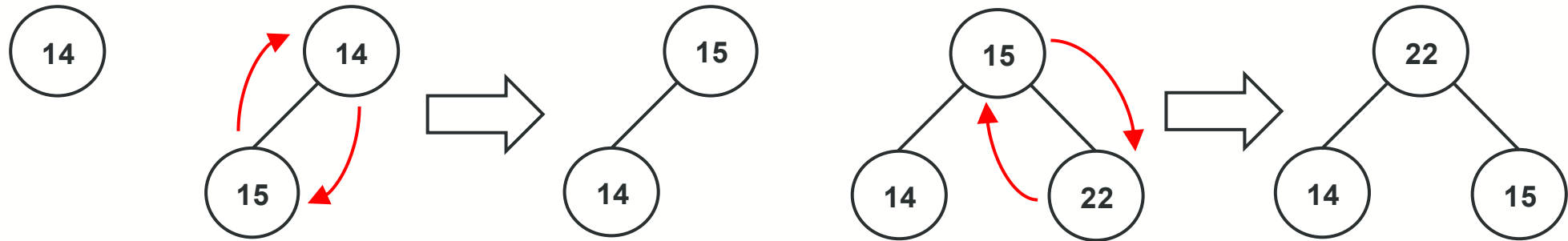


Build-Heap

- Top-down insertion ($O(n \log n)$)
 - Start with an empty heap
 - Build a heap by inserting new nodes one at a time
 - Can also be implemented in-place
- Bottom-up heapification ($O(n)$)
 - Start with the given array of unsorted items (i.e., array-representation of a binary tree without max-heap properties)
 - Start from the last non-leaf node (index $\frac{n}{2}$, where n is the array size)
 - Apply the heapify operation on each node, moving from right to left
 - Continue this process until the root node is reached

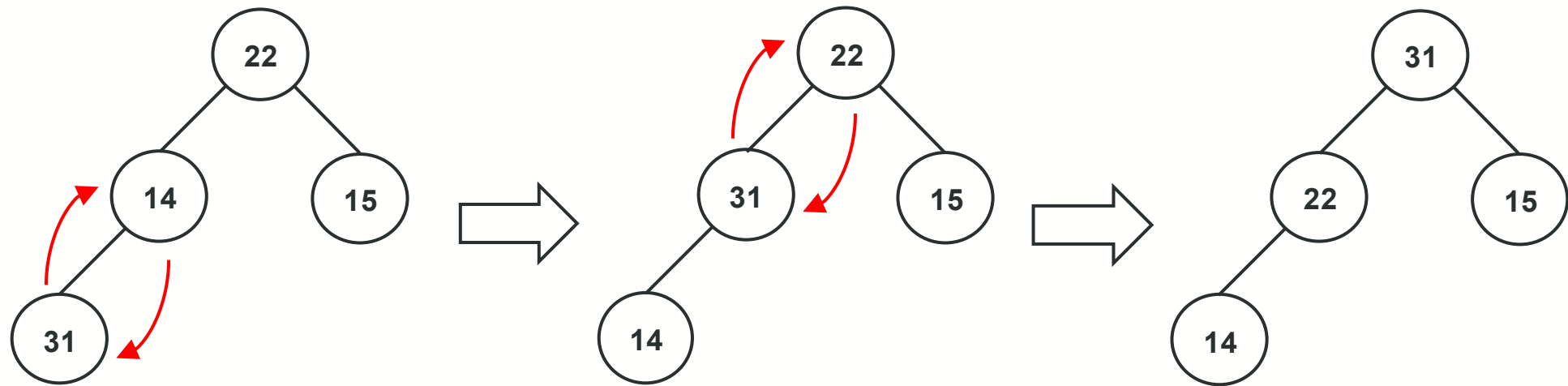
Building a Heap: Top-down Insertion (1)

A = [14, 15, 22, 31]



Building a Heap: Top-down Insertion (2)

A = [14, 15, 22, 31]



Node 15 is not affected

Building a Heap: Bottom-up Heapification

1. $\text{heap-size}[A] \leftarrow \text{length}[A]$
2. for $i \leftarrow \lfloor \text{length}[A]/2 \rfloor$ downto 1 # build heap from the bottom
3. do $\text{Heapify}(A, i)$

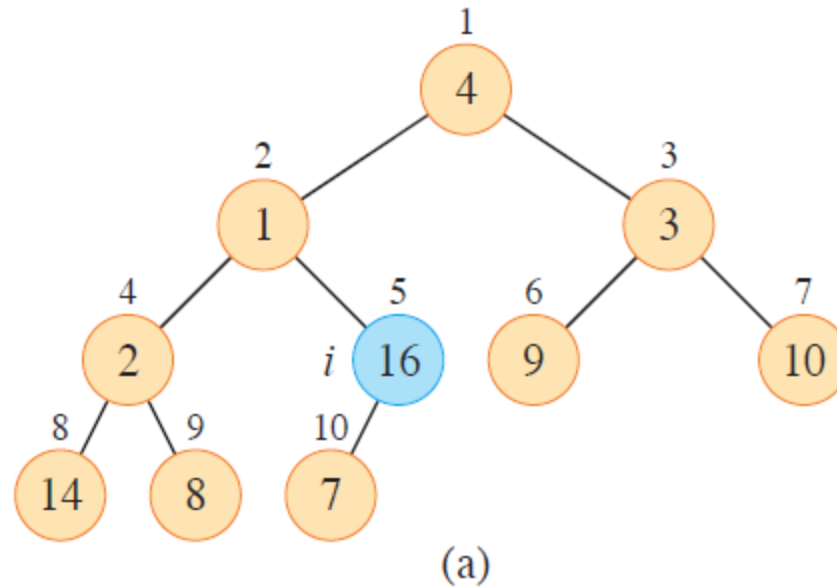
Building a Heap: Bottom-up Heapification Example

A

4	1	3	2	16	9	10	14	8	7
---	---	---	---	----	---	----	----	---	---

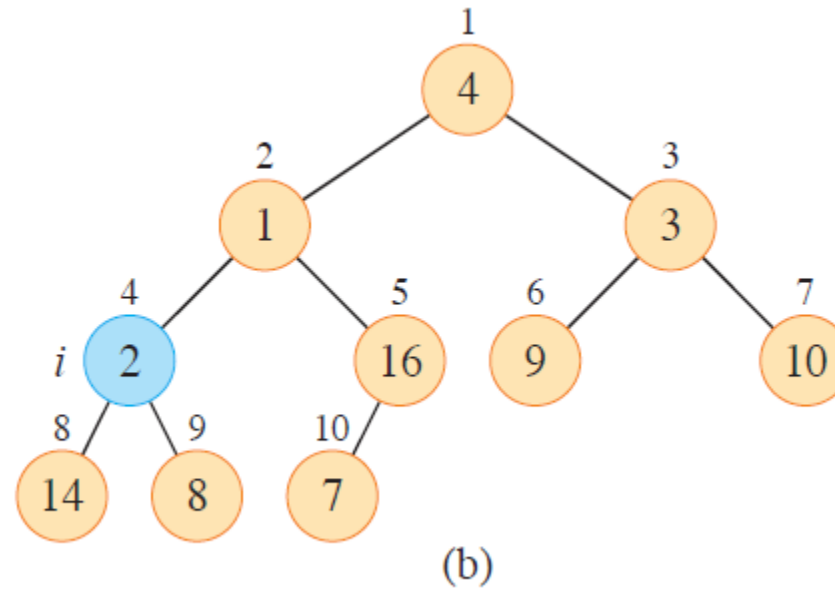
Start with the given array of unsorted items
(i.e., array-representation of a binary tree without
max-heap properties)

Building a Heap: Bottom-up Heapification Example

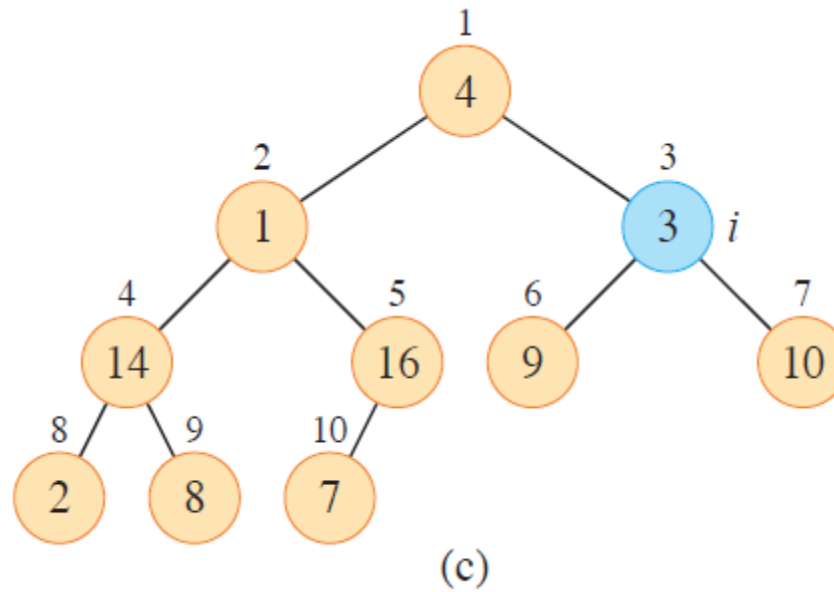


- Start from the last non-leaf node (index $\frac{n}{2}$, where n is the array size)
- Apply the heapify operation on each node², moving from right to left

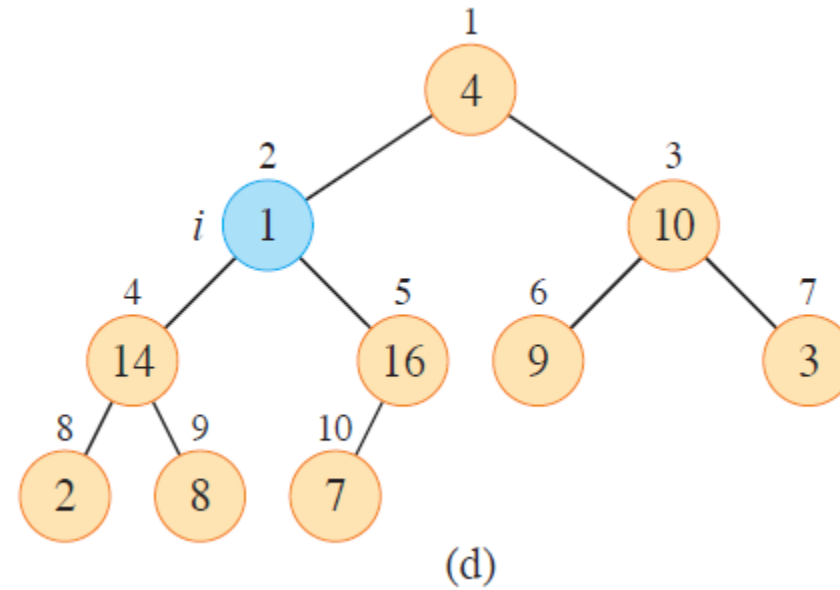
Building a Heap: Bottom-up Heapification Example



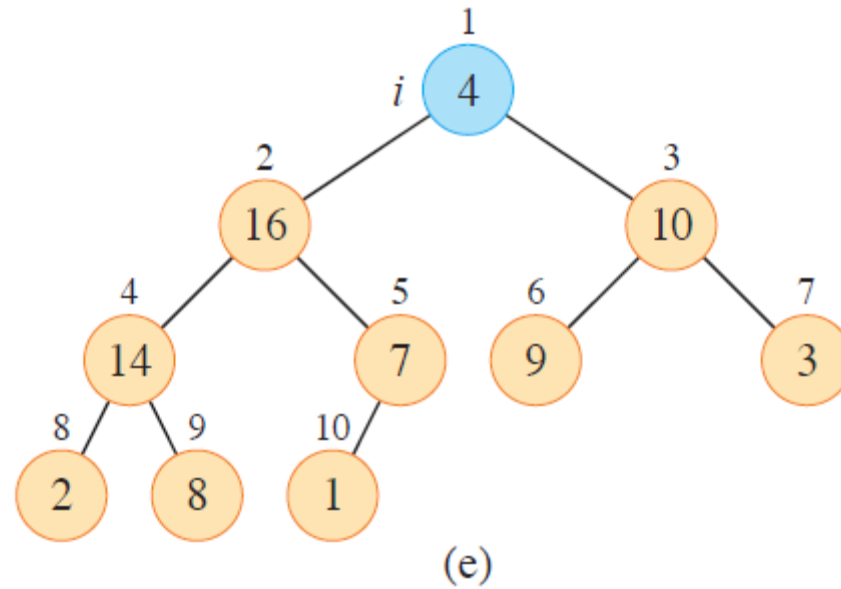
Building a Heap: Bottom-up Heapification Example



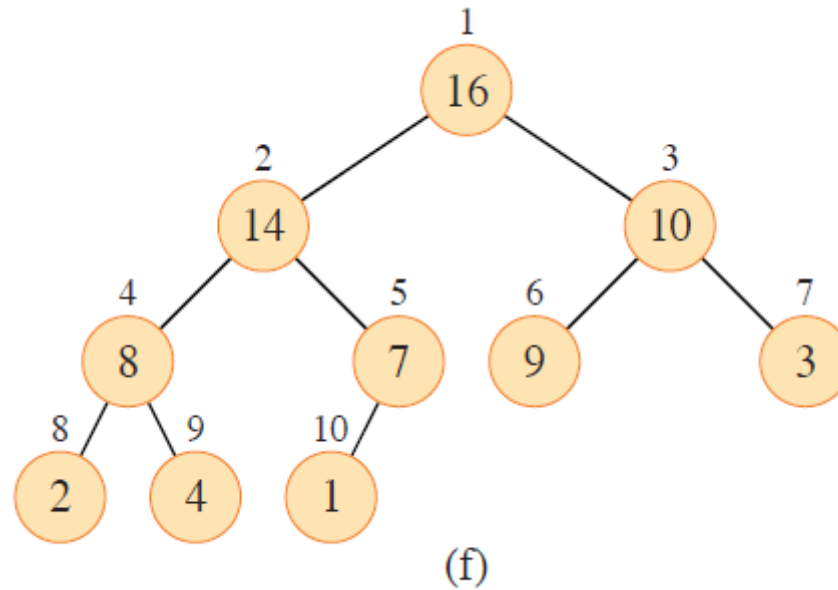
Building a Heap: Bottom-up Heapification Example



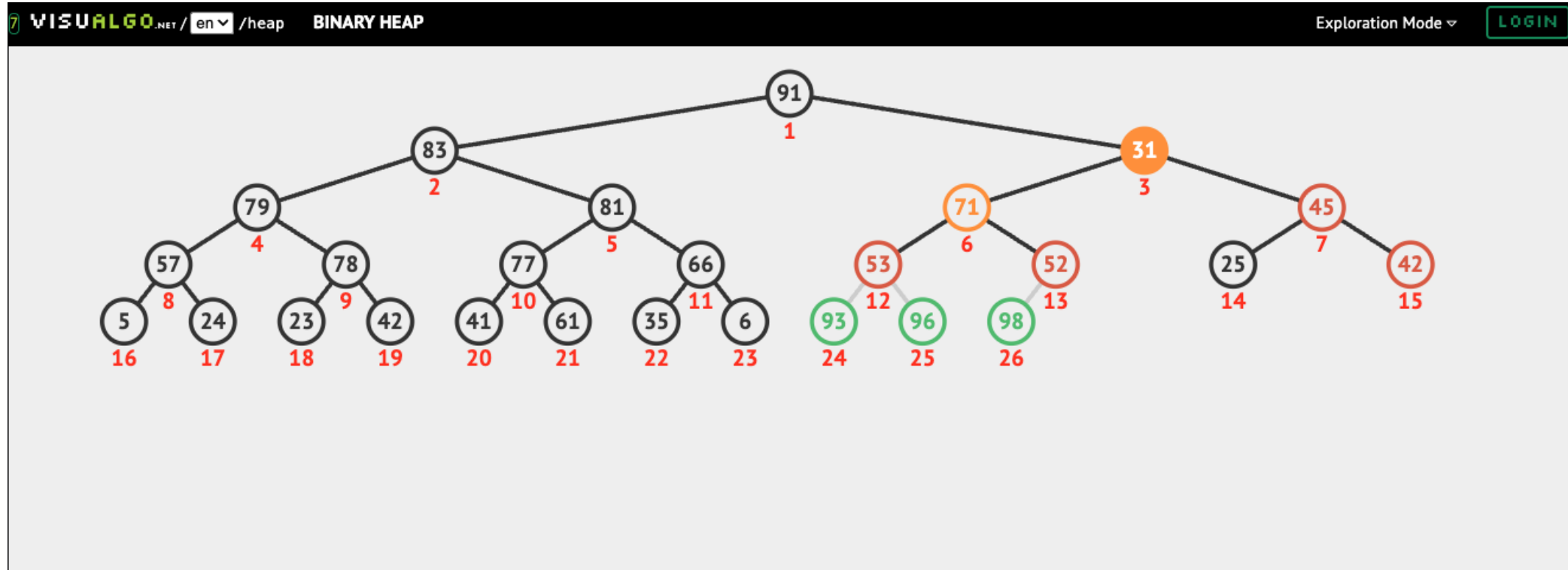
Building a Heap: Bottom-up Heapification Example



Building a Heap: Bottom-up Heapification Example



Build-Heap ($O(n)$)



<https://visualgo.net/en/heap>

Build-Heap Running Time

- Top-down insertion
 - Insert nodes $O(n)$
 - Each insertion costs $O(\lg n)$
→ $O(n \lg n)$
- Bottom-up heapification
 - Each Heapify costs $O(\lg n)$ in the worst case
 - Build-heap makes $O(n)$ calls to Heapify
→ $O(n \lg n)$?

Running Time (Tighter Bound) – not all heapify calls cost the same!

Let h the height of the tree.

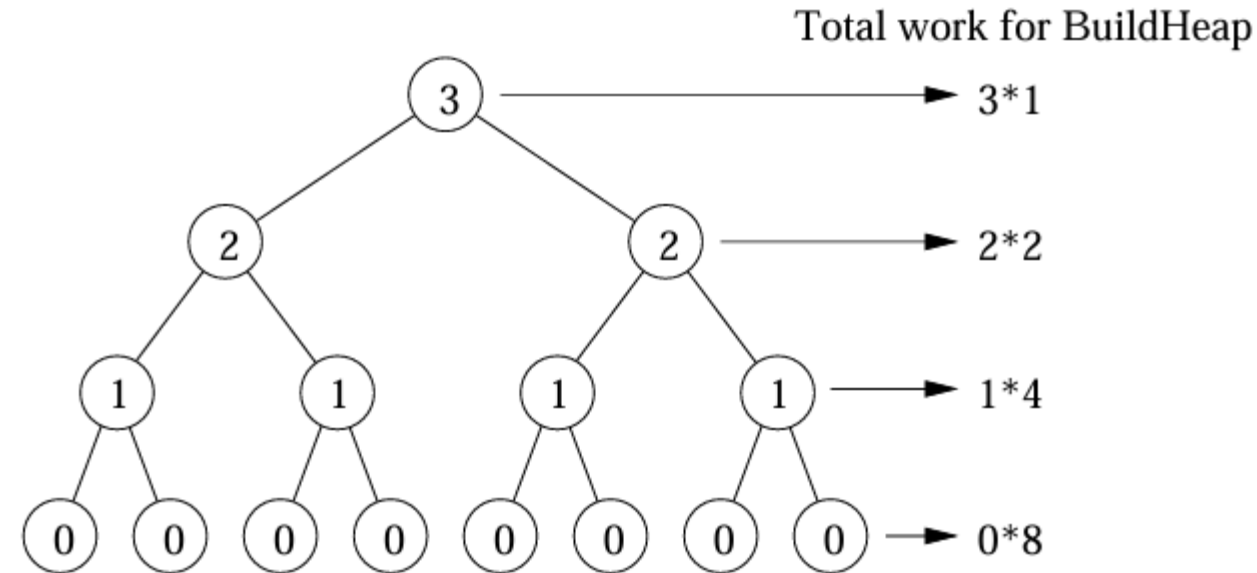
Max size of a Heap with height h is $2^{h+1} - 1$.

At height j from the bottom, there are 2^{h-j} nodes, worst case heapify: j levels

$$T(n) = \sum_{j=0}^h j 2^{h-j} = \sum_{j=0}^h j \frac{2^h}{2^j} = 2^h \sum_{j=0}^h j \left(\frac{1}{2}\right)^j$$

Using $\sum_{j=0}^{\infty} j x^j = \frac{x}{(1-x)^2} \Rightarrow \sum_{j=0}^{\infty} j \left(\frac{1}{2}\right)^j = \frac{\frac{1}{2}}{\left(1 - \frac{1}{2}\right)^2} = 2$

$$T(n) = 2^h \sum_{j=0}^h j \left(\frac{1}{2}\right)^j \leq 2^h \sum_{j=0}^{\infty} j \left(\frac{1}{2}\right)^j = 2^{h+1} = n + 1$$



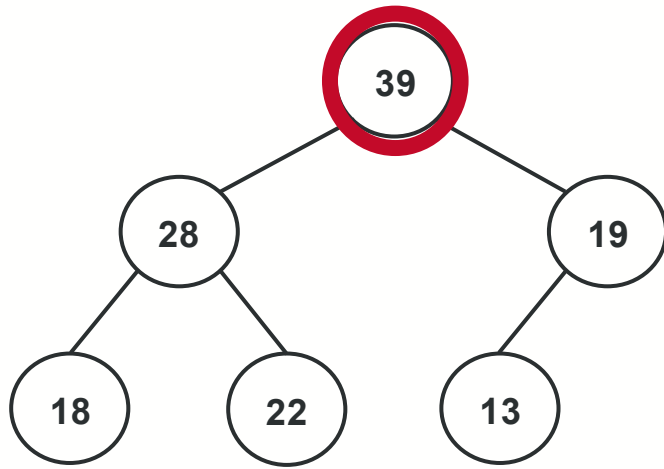
Nodes near the bottom have very small height, so heapify on them is cheap.
& most nodes are near the bottom!

Extract-Max (Delete)

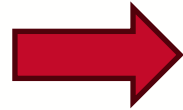
- The standard delete operation (often called `extractMax` or `deleteMax`) is designed to remove and return the maximum element, which is always at the root.

Extract-Max

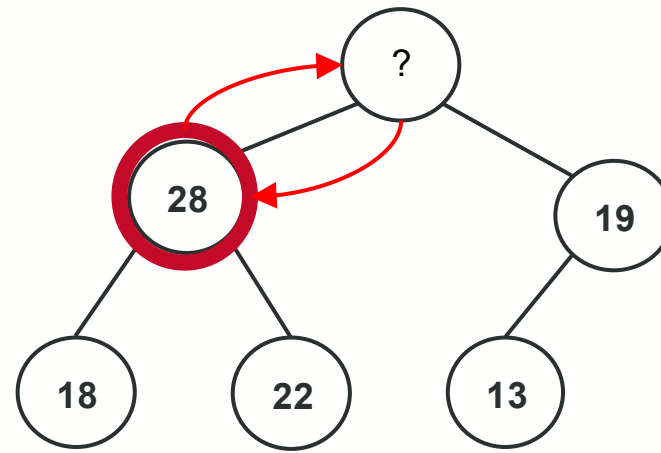
Delete root node



1	2	3	4	5	6
39	28	19	18	22	13

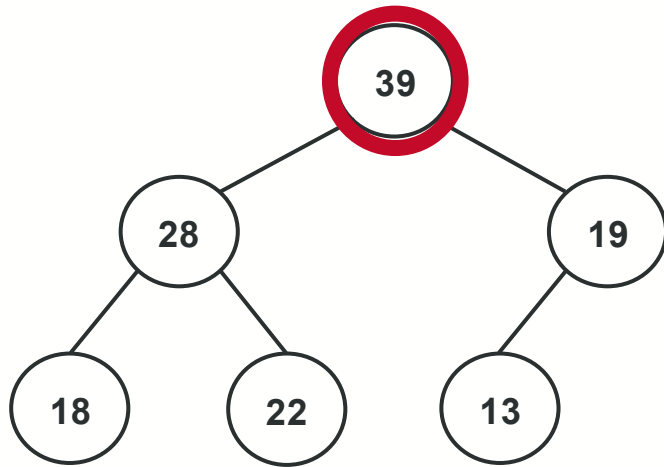


Swap down?

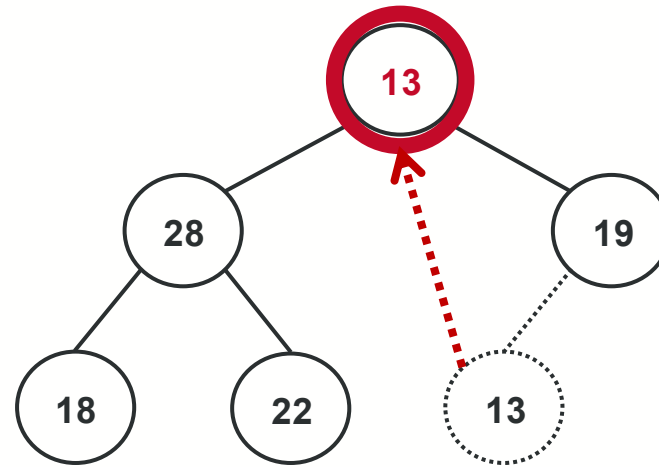
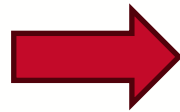


Extract-Max

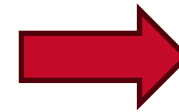
Delete root node



1	2	3	4	5	6
39	28	19	18	22	13



1	2	3	4	5	6
13	28	19	18	22	13

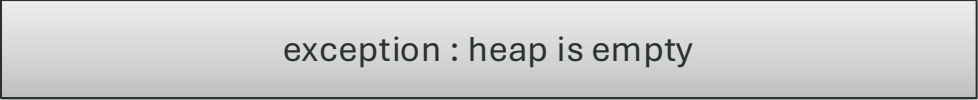


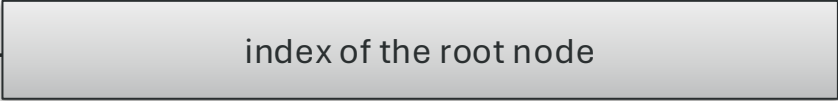
Heapify(A, 1)

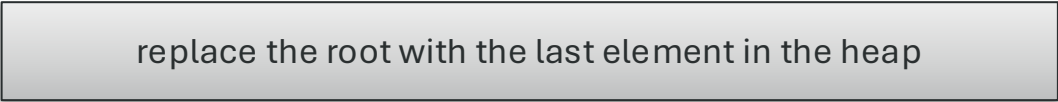


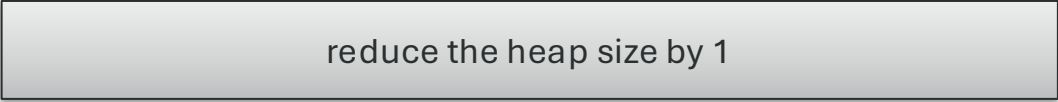
Extract-Max(*A*)

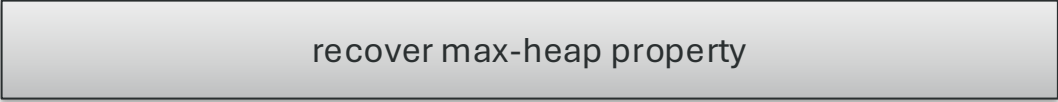
1. if $\text{heap-size}[A] < 1$

2. 

3. $\text{max} \leftarrow$ 

4. 

5. 

6. 

7. return max

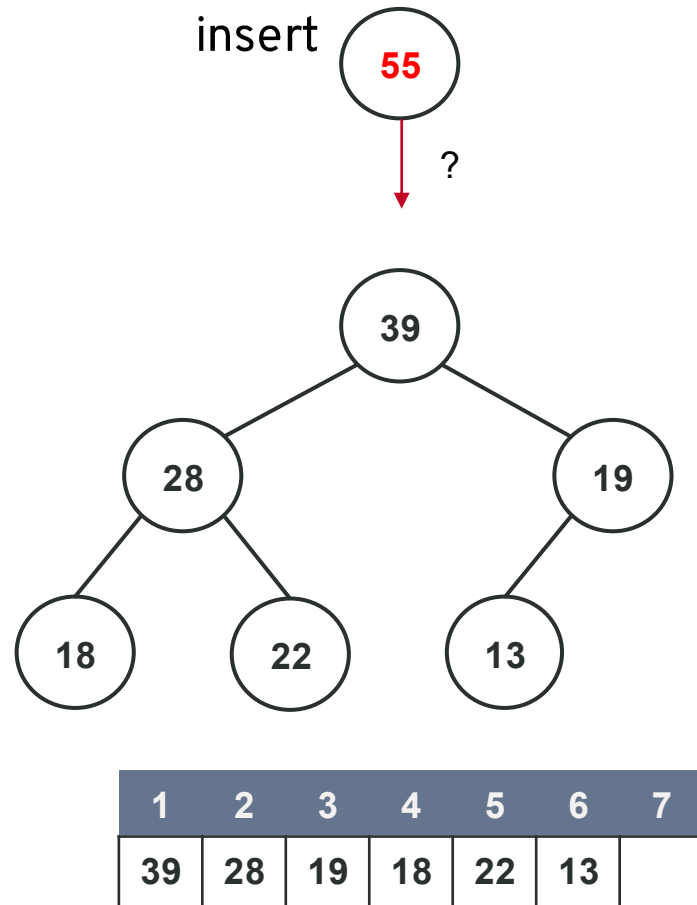
Extract-Max(*A*)

1. if $\text{heap-size}[A] < 1$
2. then error “heap underflow”
3. $\text{max} \leftarrow A[1]$
4. $A[1] \leftarrow A[\text{heap-size}[A]]$
5. $\text{heap-size}[A] \leftarrow \text{heap-size}[A] - 1$
6. Heapify(*A*, 1)
7. return max

Running Time of Heap-Extract-Max

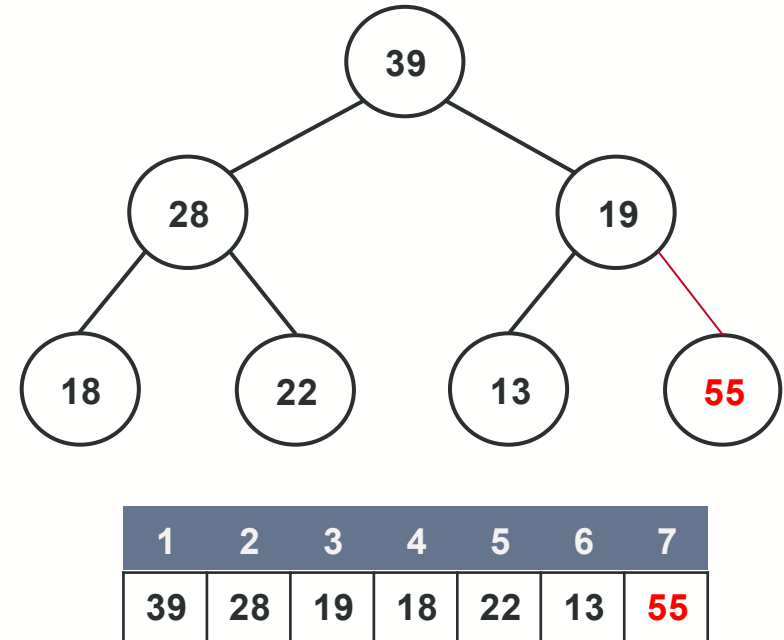
- Running time of Heap-Extract-Max is $O(\lg n)$.
 - Performs only a constant amount of work on top of Heapify, which takes $O(\lg n)$ time

Insert

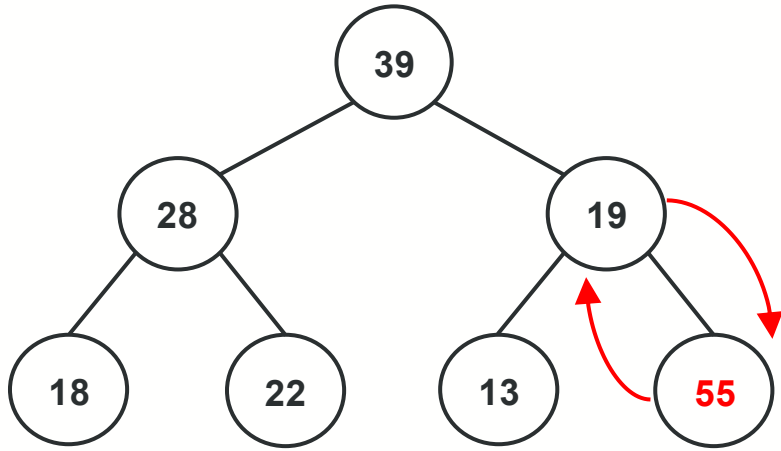


Insert

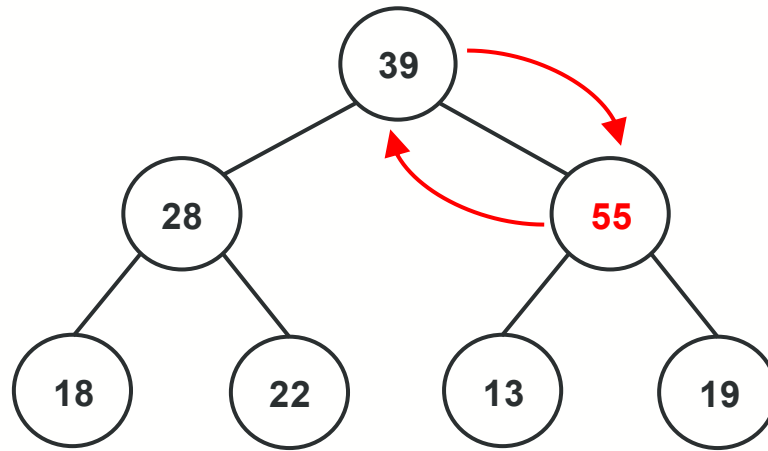
- We insert at the rightmost position to maintain the complete tree property
- Next, we fix the tree by swapping the new element with its parent, until we find an appropriate position for the element
- **Sift-up** instead of sift-down



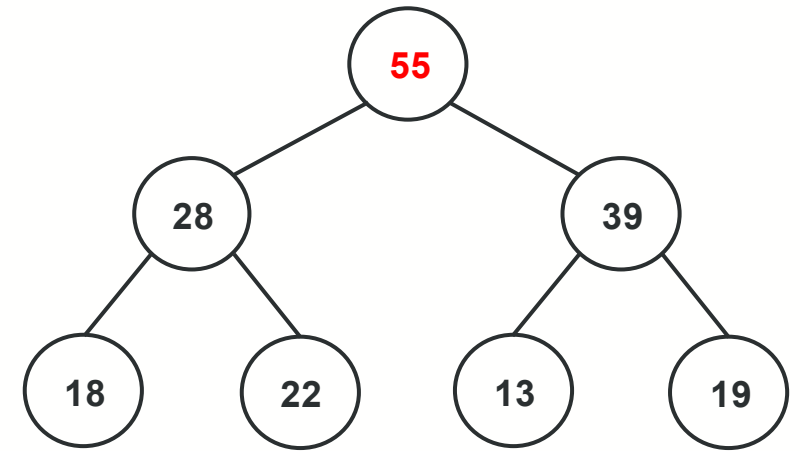
Insert



1	2	3	4	5	6	7
39	28	19	18	22	13	55



1	2	3	4	5	6	7
39	28	55	18	22	13	19



1	2	3	4	5	6	7
55	28	39	18	22	13	19

Insert(*A*, *key*)

1.

increase the size of the heap *A* by 1
2. $i \leftarrow$

last index of the heap (index of the new node)
3. while $i > 1$ and

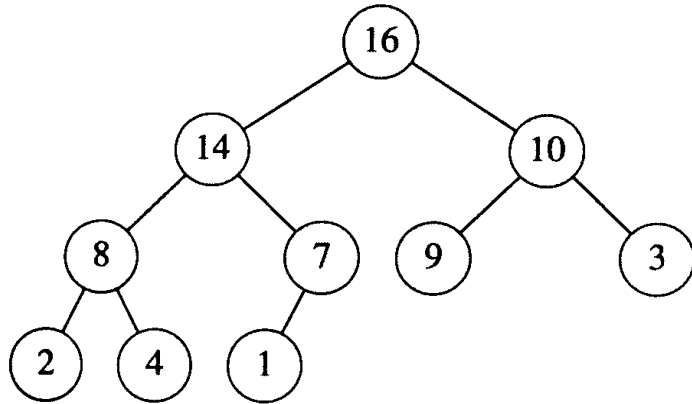
while the heap property is violated
4. do

swap i and i 's parent
- 5.
6. $A[i] \leftarrow key$

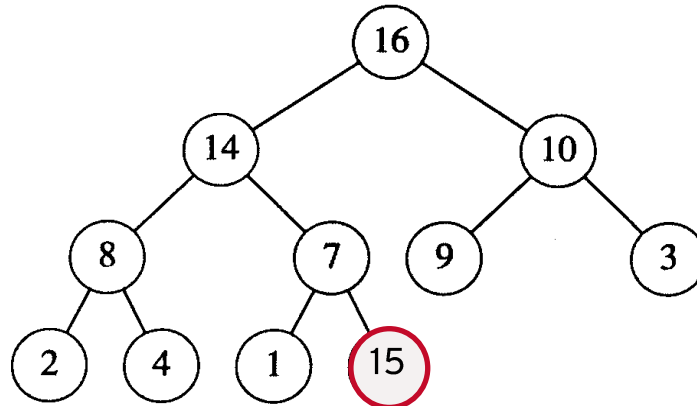
Insert(*A*, *key*)

1. $\text{heap-size}[A] \leftarrow \text{heap-size}[A] + 1$
2. $i \leftarrow \text{heap-size}[A]$
3. while $i > 1$ and $A[\text{Parent}(i)] < \text{key}$
4. do $A[i] \leftarrow A[\text{Parent}(i)]$
5. $i \leftarrow \text{Parent}(i)$
6. $A[i] \leftarrow \text{key}$

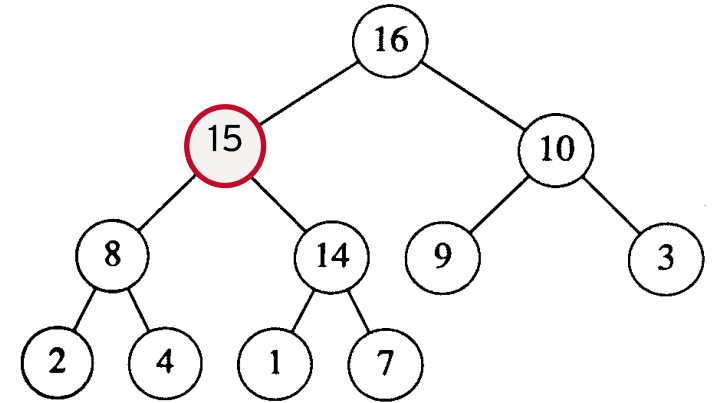
Example – Insert(A, 15)



(a)



(b)



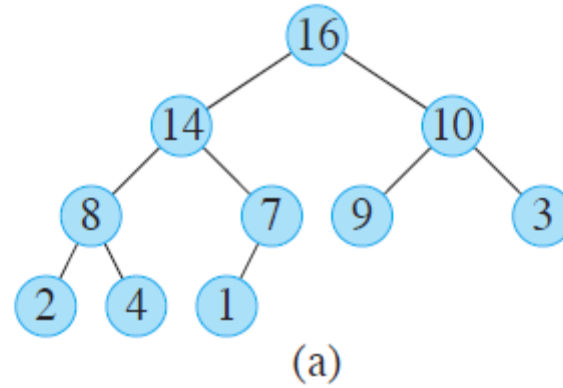
(c)

Heapsort (A)

1. Build-Heap(A)
2. for $i \leftarrow \text{length}[A]$ downto 2
3. exchange $A[1] \leftrightarrow A[i]$
4. $\text{heap-size}[A] \leftarrow \text{heap-size}[A] - 1$
5. Heapify(A, 1)

Heapsort Example

Build-Heap(A)

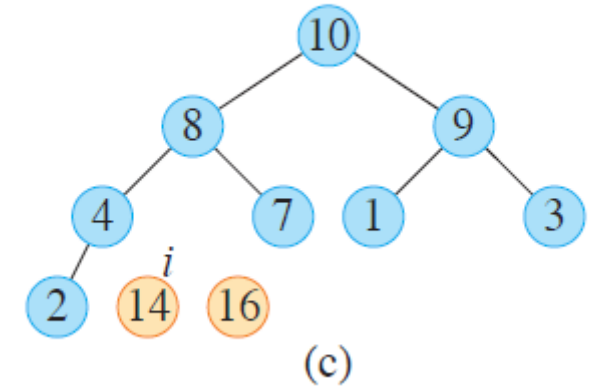
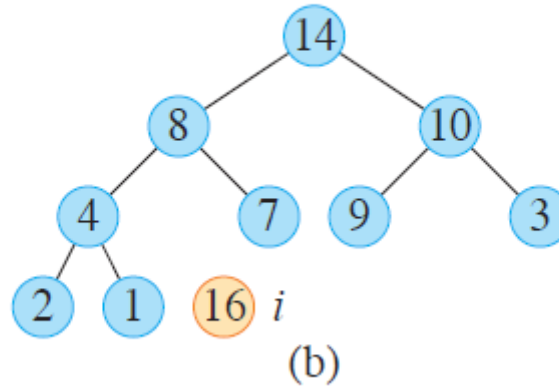
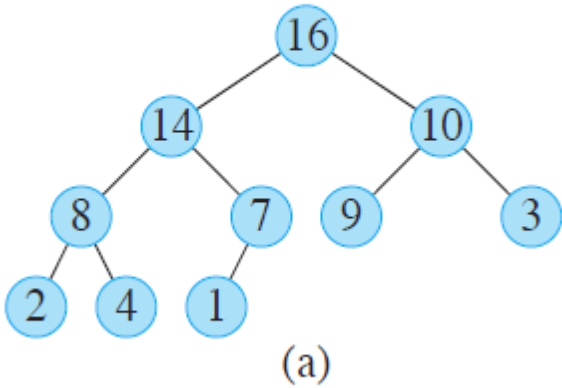


16	14	10	8	7	9	3	2	4	1
----	----	----	---	---	---	---	---	---	---

Heapsort Example

```

for i ← length[A] downto 2
  exchange A[1] ↔ A[i]
  heap-size[A] ← heap-size[A] - 1
  Heapify(A, 1)
  
```



16	14	10	8	7	9	3	2	4	1
----	----	----	---	---	---	---	---	---	---

1	14	10	8	7	9	3	2	4	16
---	----	----	---	---	---	---	---	---	----

Heapify(A, 1)

14	8	10	4	7	9	3	2	1	16
----	---	----	---	---	---	---	---	---	----

1	8	10	4	7	9	3	2	14	16
---	---	----	---	---	---	---	---	----	----

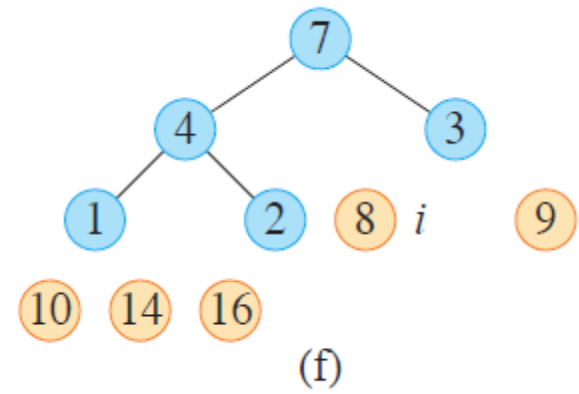
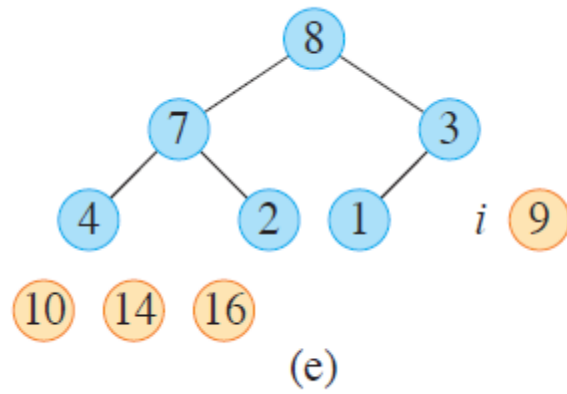
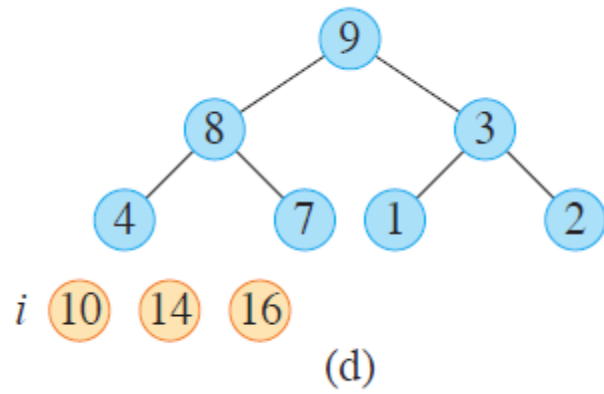
Heapify(A, 1)

10	8	9	4	7	1	3	2	14	16
----	---	---	---	---	---	---	---	----	----

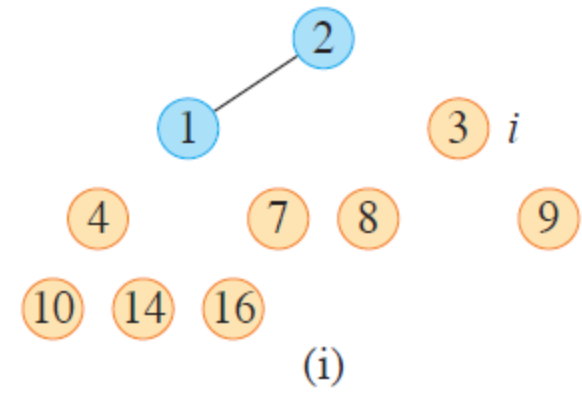
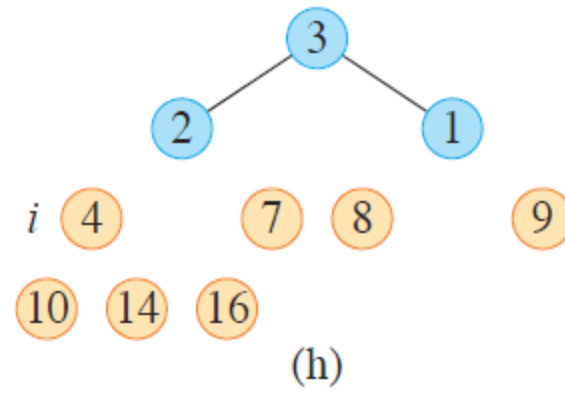
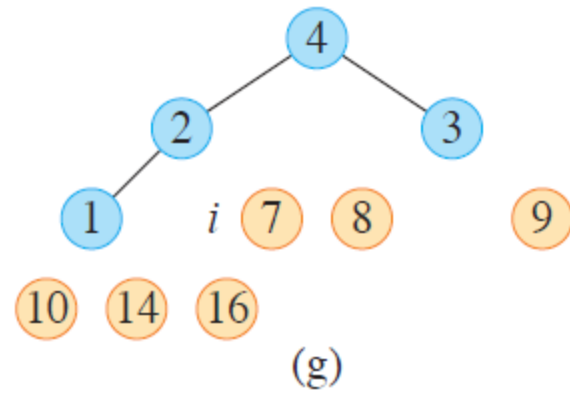
2	8	9	4	7	1	3	10	14	16
---	---	---	---	---	---	---	----	----	----

Heapify(A, 1)

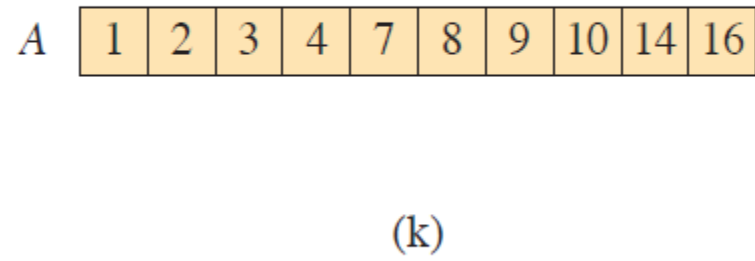
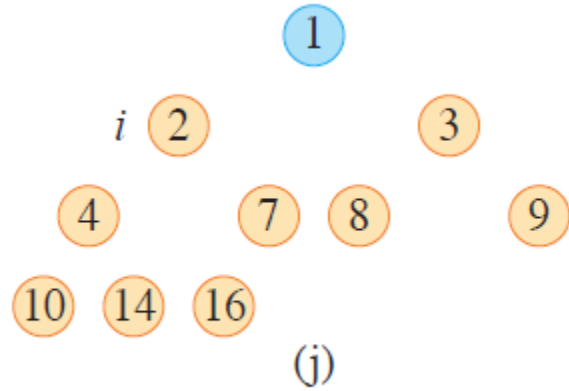
Heapsort Example



Heapsort Example



Heapsort Example



Heapsort Algorithm Analysis

- In-place
- Not Stable
- Build-Heap takes $O(n)$ and each of the $n - 1$ calls to Heapify takes time $O(\lg n)$.
- Therefore, $T(n) = O(n \lg n)$

Heap Data Structure Summary

- Construct in $\Theta(n)$ time
- Extract maximum element in $\Theta(\lg n)$ time
- Leads to $\Theta(n \lg n)$ sorting algorithm:
 - Build heap
 - Repeatedly extract largest remaining element (constructing sorted list from back to front)

Priority Queues

- A data structure for maintaining a set S of elements, each with an associated value called a key
- Unlike regular queues (FIFO), priority queues remove elements based on their priority
- Heap can be used to implement a priority queue
 - Min-Priority Queue: The element with the smallest key has the highest priority
 - Max-Priority Queue: The element with the largest key has the highest priority
- Applications: scheduling jobs on a shared computer, prioritizing events to be processed based on their predicted time of occurrence

Basic Operations

- $\text{Insert}(S, x)$ - inserts the element x into the set S , i.e. $S \rightarrow S \cup \{x\}$
- $\text{Maximum}(S)$ - returns the element of S with the largest key
- $\text{Extract-Max}(S)$ - removes and returns the element of S with the largest key

Group Activity

Streaming Top-K Selection in a Large ML Model

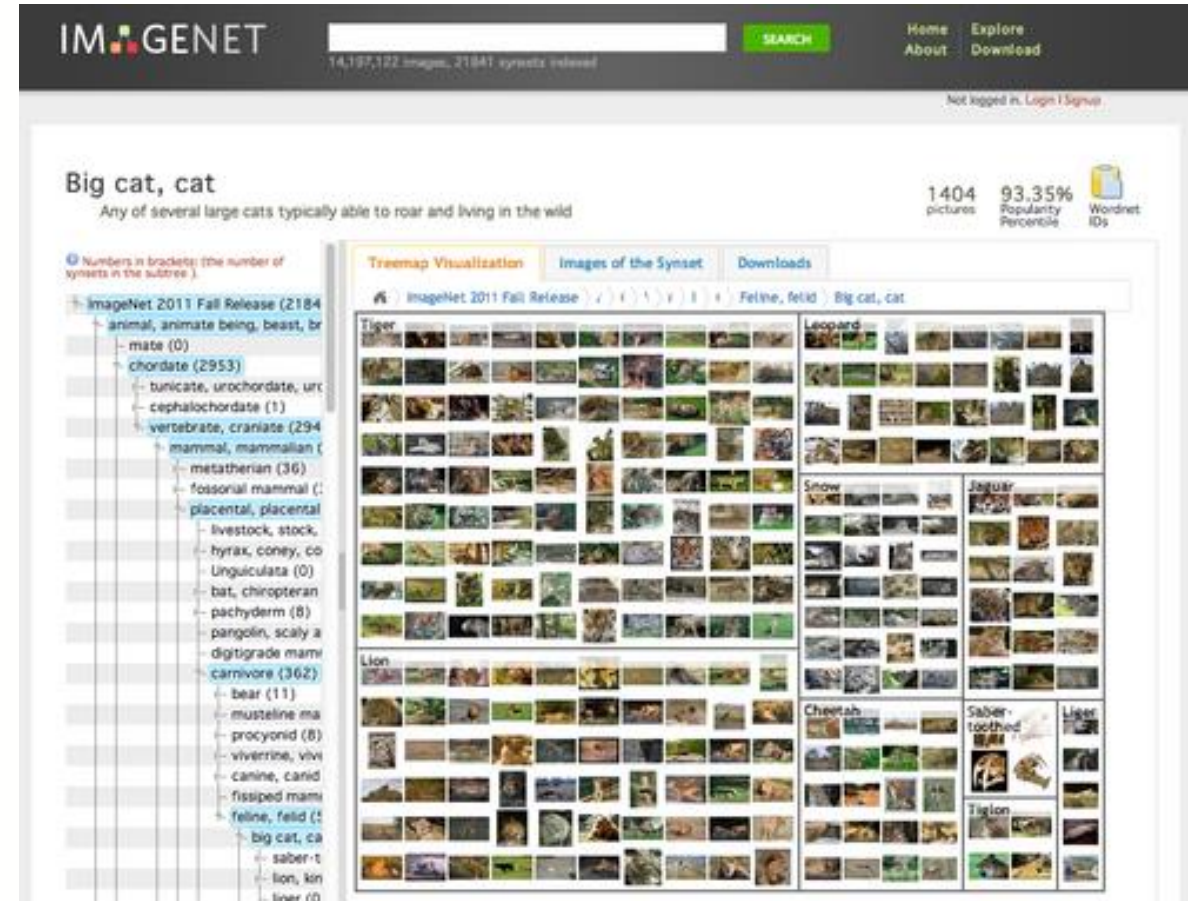
- Your model outputs 50,000 class scores per input. You need to efficiently support:
 - 50,000 inserts (one score per class): maintain
 - Extract top-k (e.g., $k=10$) highest-scoring classes
 - 100 updates, when model recalculates some scores
 - Membership checks: Is class X in the current top-k?
- Use cases: classification (top-k predictions), beam search (NLP), k-NN, gradient pruning

Example: Image Classification



ImageNet

- WordNet
 - A large lexical database of English
 - Groups words into sets of synonyms called synsets and connects them through semantic relationships (e.g., one concept)
- ImageNet-1k (ILSVRC 2012)
 - Built on WordNet synsets – each class corresponds to a WordNet concept
 - Contains 1000 classes
 - 1.2M training images
 - 100K test images
 - Evaluation metric
 - Top 5 error: the percentage of test images for which the correct label is not among the model's top 5 predictions



Which Data Structure Wins?

- Choose one data structure: Sorted array, Min-heap, Self-balanced BST (e.g., AVL, Red-Black), Hash table
- For your chosen structure, think about how fast each operation would be in Big-O terms, and in practice.
 - Streaming Inserts (50,000 scores)
 - Extract Top-k
 - Score Updates (100 recalculations)
 - Membership Check (“Is class X in top-k?”)

Work Together: Fill In the Runtime Table

Operation	Min-Heap (size K)	Balanced BST (size N)	Balanced BST (size K)	Sorted Array (size K)	Hash Table (size N)
Insert					
Peek max					
Extract top-k					
Membership check					
Update score					

Note: Evaluate the runtime for one operation (not for all 50,000 inserts or all 100 updates).

Theory vs. Practice

- Asymptotic complexity isn't the whole story
 - Typically $k \ll N$ (e.g., 10 vs. 50,000)
 - When k is tiny (like 5-20), the difference between $O(k)$ and $O(\log k)$ is negligible
 - Real performance depends more implementation details than Big-O
- Memory layout matters:
 - Heaps live in a flat array $\rightarrow$ cache-friendly, sequential memory access
 - Cache misses are often more expensive than extra comparisons
- Constant factors dominate
 - Heap operations are mostly simple swaps in an array
 - BST operations involve rotations, pointer updates, and branchy code
 - Same $O(\log k)$, but heap is faster in practice
- Modern deep learning frameworks (e.g., PyTorch, TensorFlow)
 - Often favor Parallel Sorting (e.g., Bitonic or Radix sort) or Partitioning algorithms. These algorithms exploit the GPU's massive parallelism to outperform sequential heap operations, even if their theoretical complexity is higher

Indexed Heap

- A standard binary heap is:
 - Great at: *Insert* and *Extract-Min/Max*
 - Terrible at:
 - finding an element
 - updating an element
 - checking membership
- Index Heap
 - Heap (min-heap or max-heap) + Index (hash table or a secondary array) that stores key → heap indices
 - How would this change the cost of **membership checks** and **score updates**?